

Firebase setup guide

This is a step-by-step guide for creating a **new, free Firebase project** to be the live link between the ESP32 hardware and the dashboard. It assumes zero prior Firebase experience.

Claude cannot do this part for you — creating accounts/projects is something only you can do (this is a deliberate safety boundary, not a technical limitation). Follow the steps below yourself, then see [What to hand back](#) at the end.

Why Firebase Realtime Database (not Firestore, not something else)

This isn't actually an open choice for this project — the *existing* hardware code (`hardware/original_reference/AquaGuard_full_original.ino`) already talks to a Firebase **Realtime Database** (RTDB), using paths like `/sensor/ph`, `/pumps/pump1`, `/history`. It was simply never wired to a live account during the physical rebuild. It's also still the right free choice on its own merits: free forever at this project's scale (1GB stored / 10GB downloaded per month / 100 simultaneous connections on the Spark plan — far more than one ESP32 + a few dashboard viewers need), no server of your own to run, and both the ESP32 (via the `FirebaseESP32` Arduino library) and a plain static website (via simple REST calls) can talk to it directly.

Step 1 — Create the project

1. Go to console.firebase.google.com and sign in with any Google account (a fresh one is fine — doesn't need to be tied to anything else).
2. Click **"Add project"** (or "Create a project").
3. Give it any name — e.g. `aquaguard-iot-v2`. The exact name doesn't matter, it's just a label.
4. When asked about Google Analytics: you can **turn it off** — this project doesn't need it, and skipping it makes setup one step shorter.
5. Click **Create project** and wait ~30-60 seconds for it to finish.

Step 2 — Create the Realtime Database

1. In the left sidebar, under **Build**, click **Realtime Database**.
2. Click **Create Database**.
3. Pick a region — any is fine functionally; if you're offered a choice, one physically closer to South/Southeast Asia (e.g. Singapore) will have slightly lower latency for a Bangladesh-based device, but this doesn't meaningfully matter for a sensor that updates every couple of seconds.
4. When asked to start in **locked mode** or **test mode**, choose **locked mode**. ("Test mode" leaves the whole database wide open to anyone on the internet for 30 days — we're about to set proper rules instead, so there's no need for that window at all.)
5. Click **Enable**. You'll land on the Realtime Database's **Data** tab, currently empty (just shows `null`) — that's expected, nothing has written to it yet.

Step 3 — Set the security rules

Locked mode denies everything by default. This project wants only its own known paths open (not the whole database) —

matches this project's "open writes, but scoped, not wide open" decision.

1. In the Realtime Database page, click the **Rules** tab (next to "Data").
2. Delete whatever is there and paste in exactly this:

```
{
  "rules": {
    "sensor": {
      ".read": true,
      ".write": true
    },
    "pumps": {
      ".read": true,
      ".write": true
    },
    "control": {
      ".read": true,
      ".write": true
    },
    "alerts": {
      ".read": true,
      ".write": true
    },
    "history": {
      ".read": true,
      ".write": true
    },
    "servoTrigger": {
      ".read": true,
      ".write": true
    },
    "phCalibration": {
      ".read": true,
      ".write": true
    }
  }
}
```

3. Click **Publish**.

Anything *not* listed here (i.e. the database root, or any other top-level key someone might guess) stays denied by default — Firebase RTDB rules don't inherit permissively, only these 7 named paths are actually open.

Step 4 — Get your database URL (safe to share)

1. Back on the **Data** tab, look at the top of the page — you'll see a URL that looks like: `https://<your-project-name>-default-rtdb.<region>.firebasedatabase.app` (older projects may show `https://<your-project-name>-default-rtdb.firebaseio.com` instead — both forms work the same way).
2. Copy that whole URL. **This is not a secret** — it's the equivalent of a website address, not a password. Firebase's security model puts protection in the *rules* (Step 3), not in hiding this URL. It's fine to paste this in chat, put it directly in a public GitHub repo's `app.js`, etc.

Step 5 — Get the database secret (sensitive — only the ESP32 firmware needs this)

The ESP32 firmware (added later, see the main project plan) uses the `FirestoreESP32` Arduino library's older/simpler

"legacy token" auth style, matching what's already used in `hardware/original_reference/AquaGuard_full_original.ino`. This one **is** sensitive — treat it like a password.

1. Click the **gear icon** (top-left, next to "Project Overview") → **Project settings**.
2. Go to the **Service accounts** tab.
3. Scroll down to **Database secrets** and click **Show**.
4. Copy the long string shown there.

Do not paste this into chat, a GitHub commit, or anywhere public. It goes only into your local copy of the `.ino` firmware file (as the `FIREBASE_AUTH` constant), which stays on your own computer and your ESP32's flash memory — never uploaded anywhere else.

If "Database secrets" isn't visible in your console (Google occasionally hides this for newer projects), that's fine — it just means the ESP32 firmware step later will need the slightly newer Firebase Auth (e.g. anonymous sign-in) approach instead of the legacy secret. Not a blocker at this stage; flag it when we get to the firmware step and it'll be handled then.

What to hand back when you're done

Just **one thing**, in chat:

- The **databaseURL** from Step 4 (e.g. `https://aquaguard-iot-v2-default-rtdb.firebaseio.com`)

That's enough to wire up the dashboard side later. You do **not** need to hand back the database secret from Step 5 at all — that one only ever needs to exist inside your own `.ino` file.

Also good to mention: which region you picked in Step 2 (only matters if something seems slow later, not required otherwise).